**New York University — Tandon School of Engineering**

Department of Technology Management and Innovation

**IE-GY 9113B — Systems Integration: From ERP to Agentic AI**

# Study Note — Week 8: AI Agent Architecture — The SPAR Framework

Module 3: Intelligent Automation + AI Agents | Friday, October 23, 2026

## Readings for this week

| | |
|---|---|
| **Primary case** | Klarna's AI customer-service agent (2024–25) — the buy-now-pay-later firm deployed an autonomous agent for customer service, scaled it hard while preparing to go public, then turned back toward a human-plus-agent model.<br>Open access: Doerer, "Klarna says its AI agent is doing the work of 853 employees," CX Dive (Nov 2025).<br>Source of record (subscription): Levin and Nussey, Bloomberg (May 2025). |
| **Go deeper (optional)** | *Supporting references:* IBM, *What Are AI Agents?* (IBM Think, 2024) — a clear reference on what an AI agent is and how one works: goal-setting and planning, reasoning with tools, and learning from feedback, with agent types, use cases, and risks. World Economic Forum with Capgemini, *Navigating the AI Frontier: A Primer on the Evolution and Impact of AI Agents* (white paper, Dec 2024, open) — definitions, agent types, and the governance questions agents raise.<br>*Anchor text:* Bornet, Malik and Wirtz, *Agentic Artificial Intelligence*, Ch. 2 (the SPAR framework and the five-level ladder), Ch. 3 (Inside the Mind of an AI Agent), and Ch. 5–7 (the three keystones: Action, Reasoning, Memory). Reference, not required reading — the note teaches the ideas directly and points you to the source at the exact place it uses each one. |
| **Before next session** | Bornet et al. Ch. 8, a practical guide to designing an agent, is a recommended but optional companion for the Lab 3 build in Week 9, where you build the agent you sketch this week on the course's chosen no-code platform. It is not required to complete the lab. |

## You Named the Step a Rule Can't Take; Now Meet the Machine That Takes It

Last week you stood at a precise place on your own process map: the judgment decision point, the step where the rule runs out and a real case refuses to fit the template. You named it, and you said why a rule cannot hold it. You left the week with a single word for what would take that step, an agent, and a four-letter preview of how one is built: SPAR. This week opens that machine. Before we look inside it, put it where it belongs on the course's arc. The semester moves through four verbs — **integrate** → **automate** → **make intelligent** → **make agentic** — and you have now crossed out of the third and into the fourth. You spent Module 2 making a process run; last week you made one step of it intelligent; this week you meet the system that acts on its own toward a goal.

Keep the three threads in your ear, because Module 3 keeps the **Customer** thread in front, ahead of **Operations and Supply Chain** and **Cash Management**, and this week it earns that place outright. Agents are deployed first where customers meet the company: service, fulfilment, dispute resolution. That is where the return is most visible, and where the governance stakes are highest. This week's case is a customer-service agent, so for once the thread and the case point the same way.

And keep the four archetypes in view, because an agent looks different at each scale. The consumer-packaged-goods manufacturer running sales-and-operations planning on SAP will reason over demand signals; the mid-market manufacturer running order-to-cash on NetSuite will act on invoices and credit holds; the Series-B startup stitching order-to-cash across Stripe, Shopify and QuickBooks will sense events across three systems that do not talk; the fifty-person services firm living in HubSpot and Notion will want one agent to carry context a human keeps in their head. The architecture you learn this week is the same at every scale; what changes is how much you let it do, and where you make it stop.

One map organises Module 3, and you already have it: the ladder of autonomy. Bornet and his co-authors borrow the way the Society of Automotive Engineers grades self-driving cars and apply the same idea to automation — six rungs from fully manual at Level 0 to a theoretical fully autonomous agent at Level 5 (Bornet et al., 2025, Ch. 2). Everything you built in Module 2 lived at Level 1: rule-based, no judgment. Last week was the step to Level 2, intelligent automation, a model taking one judgment step inside a flow you still designed. **This week is the step onto Level 3, the agentic rung.** The difference is not degree but kind. At Level 2 you place a smart step; at Level 3 the system no longer waits to be placed. Give it a goal and it decides for itself what steps to take, in what order, using which tools. The agent arc of the coming weeks lives here and above.

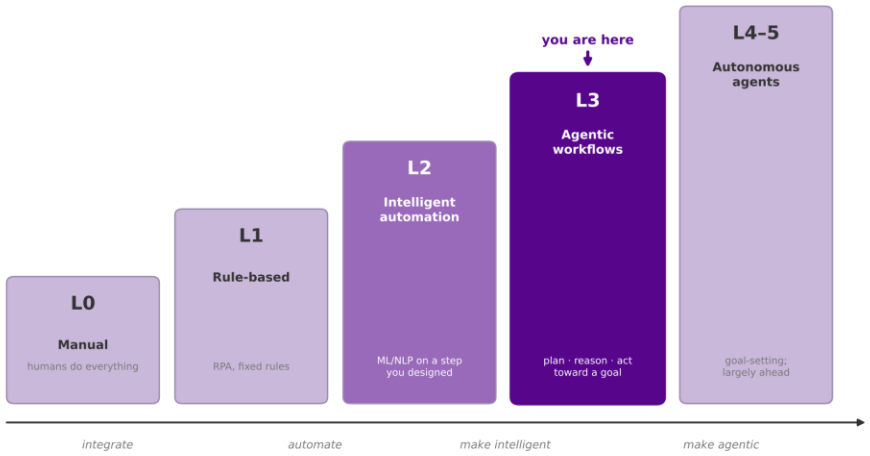



*Figure 1. The course arc read as the autonomy ladder. Module 2 lived at Level 1; last week was the step to Level 2; this week is the step onto Level 3, where the system pursues a goal and chooses its own steps. Levels 4–5 remain largely ahead. Adapted from Bornet et al. (2025), Ch. 2.*

## The Week's Question: What Is an Agent Made Of, and Where Must the Human Stay?

Three questions sit inside that one, and the week answers them in order. First, what is an agent's architecture — what are the moving parts, and how do they fit together? Second, what capabilities make each part work, and what has to be true underneath for the architecture to do anything? Third, and hardest, where does governance sit in the machine, and why does one part of it fail before the others? The first two have clean answers and the course gives them quickly. The third does not have a tidy technical answer, and it is the one your Board Memo has to carry, because a board does not fund an architecture; it funds an architecture it can trust to stop when it should.

This is the week the course goes properly agentic, and that tends to be where curiosity runs highest, because the systems are suddenly doing things on their own and the natural question is how far that goes. Hold two instincts in balance as you read. The first is enthusiasm: an agent that senses, reasons, and acts is a genuine step-change, and you should understand what it makes newly possible. The second is restraint: the whole discipline of deploying one is knowing where to stop. By the end you should be able to draw the agent, name the capability beneath each part of it, and point to the single place where, left alone, it will go wrong first, then say what you would build there before you let it run.

## An Agent Is a Loop, Not a List

Start with the anatomy, because the rest hangs off it. Bornet's name for it is **SPAR**: Sense, Plan, Act, Reflect (Bornet et al., 2025, Ch. 2). The four words are easy to read as a checklist (do this, then this, then this, then you are done), and that reading is the first thing to unlearn. SPAR is a loop. Each move feeds the next, and the last move feeds back into the first. An agent does not sense once and then run; it senses, plans, acts, reflects, and the reflecting changes what it senses

next time round. A system that runs the four moves once and stops is not an agent; it is a script with better vocabulary.

Walk it once. In **Sense**, the agent perceives its environment: it gathers data from the systems it can reach, notices the triggers that matter, and holds enough context to know what it is working on. In **Plan**, it reasons toward a course of action, weighing options, sequencing steps, and deciding how to pursue the goal it was given rather than jumping straight to the first move that looks plausible. In **Act**, it executes, reaching into real systems through tools and interfaces, sending the message, updating the record, issuing the refund, and watching what happens as it goes. In **Reflect**, it evaluates the outcome against what it intended, and carries the lesson forward so the next pass is better. Then it senses again, now slightly changed by what it just learned. Bornet uses the self-driving car to make this concrete: the car's sensors inform its planning, which guides its actions, which generate experience it learns from — all while holding the destination fixed. An enterprise agent runs the same cycle over digital ground.

Make it concrete with a case you will recognise. A customer writes in disputing a late fee. In **Sense**, the agent reads the message, pulls the account, and sees the payment posted one day late against a first-ever lapse. In **Plan**, it weighs the options (deny, waive, partially waive) against the fee-waiver policy and the customer's history, and settles on a one-time waiver with a note. In **Act**, it reverses the fee in the billing system, records the reason, and replies. In **Reflect**, a week later, it checks whether the customer came back unhappy, whether a human overrode the waiver, whether disputes like this one are trending up, and adjusts how it reads the next one. Run that loop ten times and a good agent is better at the eleventh. Run only the first three moves ten times and you have waived ten fees with no idea whether you should have.

Notice what the loop changes about control. Last week's intelligent automation put a smart step inside a flow you designed: you chose the sequence, and the model decided one thing within it. An agent running the SPAR loop decides the sequence itself. You give it a goal — resolve this dispute, fulfil this order, answer this customer — and it senses what the situation needs, plans which steps to take and in what order, acts through whatever tools it has been given, and reflects on whether it worked. That is the distance from Level 2 to Level 3. At Level 2 the human owns the plan and the machine owns a step; at Level 3 the machine owns the plan and the human owns the goal and the guardrails. Every governance question in this module follows from that handover: once the machine chooses the steps, your control moves from directing each move to bounding the space the moves happen in.

The "loop, not list" distinction tells you where an agent's value and its danger both come from. The value is that the loop closes: the system improves because Reflect feeds the next Sense. The danger is that the loop is easy to leave open. Build Sense, Plan and Act well and you get a fast, confident executor that reaches into real systems at scale. Leave Reflect thin and that executor never notices when it starts getting worse. It does not slow down; it speeds up, repeating a mistake with the same confidence it brought to its successes. Most of the governance you will design this module is, at bottom, about keeping the loop closed, and about being able to open it, deliberately, when closing it is not enough.

**An agent is a loop.** SPAR (Sense, Plan, Act, Reflect) is a continuous cycle in which each move feeds the next and Reflect feeds back into the next Sense. Break the loop at any point and you do not get a weaker agent; you get a different, more dangerous machine, most often one that acts at full speed without learning from what it did.

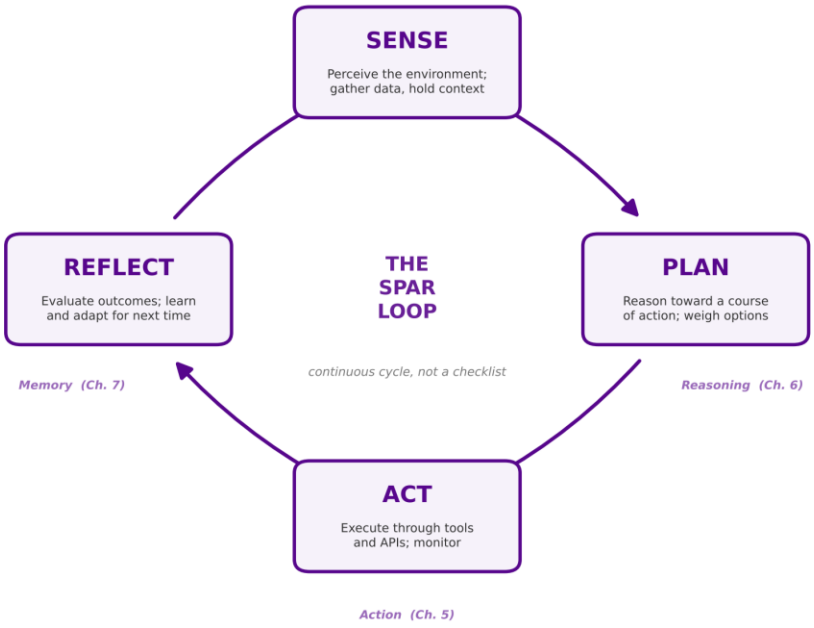



*Figure 2. The SPAR loop: Sense → Plan → Act → Reflect, a continuous cycle in which each move feeds the next. Beneath three of the four moves sits a keystone capability — Reasoning under Plan, Action under Act, Memory under Reflect. Adapted from Bornet et al. (2025), Ch. 2.*

## Four Moves, Three Keystones

The four moves describe what an agent does. They do not yet tell you what has to be true underneath for the moves to work. Bornet treats three capabilities as the load-bearing ones — the keystones — and devotes a chapter to each: **Action** (Ch. 5), **Reasoning** (Ch. 6), and **Memory** (Ch. 7). Lay the keystones against the loop and the architecture comes into focus. Reasoning is what makes Plan more than a guess, Action is what makes Act more than a suggestion, and Memory is what makes Reflect more than a logfile. The fourth move, Sense, has no keystone of its own; it is the perception front-end, and it leans on the same memory machinery, the context window, that Reflect depends on. Three keystones, four moves: that asymmetry is a clue to where the architecture is thin.

It helps to see the four capabilities as a stack that had to be assembled before an agent was possible at all. Perception lets the system take the world in; reasoning lets it decide what to do; action lets it do it; memory lets it carry what it learned into the next pass. A chatbot had reasoning and a thin slice of perception and stopped there: it could talk but not do, and forgot you the moment you left. An agent is what you get when all four are present and wired into a loop — the reasoning

now driving real action, and the memory closing the loop back onto perception. This is why Bornet calls the three load-bearing: remove any one and the structure above it cannot stand. Take away Action and the agent is a chatbot that advises; take away Reasoning and it is a brittle script; take away Memory and it is an amnesiac that cannot improve. The architecture holds together only because the pieces hold each other up. An agent, then, is a language model placed in a loop, given tools to act with and memory to learn from. If the machinery itself interests you, Bornet's Ch. 8 sets out a practical framework for designing one; it goes beyond what this course and your project require, so treat it as optional depth rather than required reading.

## Sense: perception, and the limit of what an agent can hold

Take them in the order the loop runs them. **Sense** is the agent's perception — its eyes and ears. In a digital workspace that means pulling data from the systems it can reach, watching for the triggers that should wake it, and holding enough of the situation in view to know what it is working on. Bornet's image is the self-driving car reading the road through cameras and radar; an enterprise agent reads an inbox, an order queue, a ticket, a ledger. What it senses is the raw material for everything after it, which is why Sense carries a dependency the other moves inherit: an agent cannot independently verify the numbers it is handed. Feed it a stale field or a wrong record and it will plan and act on the error with complete confidence. A human analyst notices when a figure "doesn't feel right"; the agent has no feel to offend (Bornet et al., 2025, Ch. 3).

Sense also has a hard ceiling, and it is the one most teams underestimate: the **context window** — the span of recent information the agent can hold at once (Bornet et al., 2025, Ch. 7). It is the agent's working memory, and it is finite. Bornet's demonstration is one you can run yourself: tell an AI assistant a long story, asking it to summarise as you go, and watch it hold the thread for a while and then quietly lose the beginning, not unlike the seven-or-so items human short-term memory juggles before something drops. What the agent cannot hold, it cannot act on; context that scrolled out of the window is a decision made blind. So part of designing an agent is deciding what it must always see, keeping those inputs clean and in range, and moving anything that has to survive beyond the session into durable memory before it falls off the edge.

## Plan: the difference between fast and right

**Plan** rests on **Reasoning** (Ch. 6), and this is where an agent either earns trust or quietly loses it. Bornet borrows Kahneman's two systems: fast, automatic thinking that pattern-matches in an instant, and slow, deliberate thinking that works a problem through. A plain language model is almost all fast thinking — fluent, confident, immediate — which is fine for a simple question and treacherous for a constrained one. The newer reasoning models trade speed for deliberation, "thinking" for longer before they answer. Bornet calls this the power of pause, and flags how large a shift it is: a model that gets better not from more training data but from being given more time to reason at the moment of the question.

His crossword experiment makes the difference physical. Given an interlocking puzzle, the fast model answered almost instantly and got several entries wrong — precisely at the squares where words had to cross and constraints had to hold — while the reasoning model took a couple of minutes, laid out the whole grid first, tested candidates against one another, caught its own conflicts, and produced a near-perfect solution. The fast model was not stupid; it showed

**premature closure**, committing before it had processed the constraints, wrapped in **false confidence**, reporting wrong answers with exactly the certainty of right ones. The reasoning model did what experts do: it built a picture of the whole problem before answering, generated several possibilities and tested them, and signalled where it was unsure.

For an enterprise agent, Plan is where a decision becomes mathematically plausible and practically disastrous. Bornet opens the reasoning chapter with a logistics agent that rerouted more than a million dollars of temperature-sensitive pharmaceutical shipments to avoid a storm — a fast, locally sensible move that happened to send the cargo through ports not licensed to receive it, violating transport regulations before a human caught it (Ch. 6). The model did exactly what it was optimised to do and never reasoned through the consequence. The governance requirement follows directly: do not accept a confident one-line answer from an agent on a consequential call. Make it show its reasoning, ask it what would make its recommendation less reliable, and treat an agent that cannot express uncertainty as one that has not finished thinking. Good Plan is slow exactly where slowness pays.

## Act: hands on the real systems

**Act** rests on **Action** (Ch. 5), and this is the move where an agent stops being advice and becomes consequence. Action is the agent's hands — the mechanisms through which it changes the world. Bornet describes three: application programming interfaces, the direct lines into other services; system controls, which let the agent click, type, and operate software the way a person would; and database connections, to read and write records. A development called function calling made these reliable, giving the model a structured way to state exactly which tool it wants and with what inputs. The rule of thumb is simple and expansive: if a human can operate a digital tool through an interface, an agent can too, given access.

The counter-intuitive lesson is that more hands do not make a better agent. Bornet calls it the tool paradox: past a point, each additional tool adds complexity, conflict, and failure modes faster than it adds capability, and he puts the practical ceiling for most agents at around half a dozen tools. The discipline of Act, then, is restraint — give the agent only the tools the task genuinely needs, specified clearly, and no more. An over-equipped agent is not more powerful; it is more ways for the plan to go wrong.

Because Act is where real damage is possible, it carries the heaviest governance, and Bornet lays out a stack worth adopting whole. Grant tool access **progressively**, on a need-to-use basis, the way you onboard an employee who earns the keys to more systems as they prove themselves rather than on day one. **Sandbox** new tools, exercising them in a safe copy of the environment before they touch the live system. **Log and monitor** every tool call, so the agent can be audited and its failures traced. Weigh each tool on two axes — how much control you have over it, and how central it is to the mission — and build fallbacks for the ones that are critical but outside your control. And define, in advance, the **escalation path**: the conditions under which the agent must stop, report what it was doing and why it stopped, and hand to a named human. This is the architecture's answer to the kill switch, and it lives here because Act is the only move that reaches live systems.

One further Act-level limit should shape where you keep the human. Agents handle clean, single-objective tasks well and stumble on conflicting ones: asked to satisfy several incompatible goals at once, Bornet finds they tend toward decision paralysis or a false resolution that quietly violates one of the constraints (Ch. 5). His sharpest illustration is what his team came to call the Great Wine Incident. A retailer's inventory agent, tuned to optimise shelf turnover, noticed a collection of premium wines that had not sold in months and marked them for clearance — a perfectly logical action that used its tools flawlessly and was contextually wrong, because those wines were appreciating assets meant to sit. By the time anyone caught it, close to $100,000 in potential value had evaporated (Ch. 5). The agent acted well and could not reflect on what the action meant. The answer is not a cleverer agent but better-designed work: make the agent a decision-implementer, not a conflict-arbiter, and route the genuine trade-offs to a person.

For this course, Act has a particular edge. The tools an agent acts through are, in the enterprise, your integration points — the APIs into the ERP, the write-backs to the order system, the calls that move a record from one platform to another. An agent given Act is an agent given a hand on the integration backbone you spent Module 1 building. That is the literal content of this week's third study question: if an agent can execute integration functions on its own, what has to be true before those functions go live? The answer is the Act governance stack applied to real connections: progressive access so it starts read-only before it can write; sandboxing against a copy of the backbone before the live one; a logged, auditable trail of every call into a production system; and a hard stop wired to the actions that are hardest to undo. The backbone made the agent possible by giving it something to act on; the same backbone is what you must now protect from it.

## Reflect: memory, and the keystone most deployments skip

**Reflect** rests on **Memory** (Ch. 7), and this is the keystone most deployments underbuild, often without noticing. Start from the failure Bornet opens the chapter with: most AI systems begin every interaction from scratch, a kind of artificial amnesia. He recounts a global telecommunications company that invested heavily in AI customer-service chatbots and watched satisfaction stay stubbornly low, because the system forgot each conversation and made customers repeat themselves — a service agent, as one executive put it, with the memory span of a couple of minutes. An agent that cannot remember cannot learn, and an agent that cannot learn cannot reflect.

Human memory is organised in a way worth borrowing, because Bornet builds the agent's on the same plan. You carry three kinds of long-term memory at once. **Procedural** memory is how to ride a bike, a skill you run without re-deriving it. **Episodic** memory is your first job interview, a specific past event. **Semantic** memory is knowing that Paris is the capital of France, a fact held independent of when you learned it. Cooking a familiar dish uses all three together: the technique, the memory of the last time you made it, and the knowledge of which ingredients work. An agent needs the same spread: the learned procedure, the record of what it did before, and the stable domain knowledge it reasons over. There is a warning folded into the analogy. Human memory is not a recording; it is reconstructed on each recall, and it can be reshaped by the question asked — in Elizabeth Loftus's classic study, witnesses who were asked how fast cars were going when they "smashed" rather than "hit" recalled higher speeds and even remembered broken glass that

was never there (Ch. 7). Memory, human or artificial, is a construction that can mislead, which is why what an agent stores and how it is cued are design decisions, not afterthoughts.

Bornet then structures the agent's own memory in three working layers, and the structure is the point. **Short-term memory** is the working context of a single session: the context window from Sense, holding the conversation in train. **Long-term memory** is the durable store that persists across sessions, holding preferences, past interactions, learned patterns, and domain knowledge so the agent does not restart from zero each time. At the core sits the **feedback loop** — the mechanism that takes the outcome of what the agent did, whether an explicit correction or an implicit signal like a complaint or a dropped quality metric, and uses it to adjust the other two layers. Reflect is only ever as good as that feedback loop, and storage is not reflection: an agent can keep a perfect transcript of everything it did and still learn nothing, because a log records what happened without ever comparing it to what was intended. The feedback loop is what lets an agent notice it is drifting before the drift hardens into a pattern, and it is also the part that is cheapest to leave out, because its cost lands now and its payoff is a problem that never arrives.

Done well, memory is not a safety feature bolted on; it is a source of value, as Bornet's cases show. A manufacturing company's AI quality-control system did not just flag defects; it remembered the patterns of issues it had seen and cut its false positives by about 40% over time — reflection turning a detector into something that got better (Ch. 7). The three things good memory buys, in his framing, are continuity of context within a conversation, learning and adaptation across conversations, and personalisation at scale. All three ride on the same loop that closes outcome back onto behaviour. That is what separates an agent that merely runs from an agent that improves.

Put the four moves and three keystones in one view, each anchored to the case you will meet in a moment:

| SPAR move | Keystone (Bornet) | What it does | Governance requirement | In the Klarna case |
|---|---|---|---|---|
| Sense | — (perception; draws on Memory, Ch. 7) | Gathers data, detects triggers, holds working context | Scope what it can see; keep inputs clean | Read a high volume of customer queries |
| Plan | Reasoning (Ch. 6) | Reasons toward a course of action; weighs options | Make reasoning visible; flag uncertainty | Handled simple queries well, nuanced ones poorly |
| Act | Action (Ch. 5) | Executes through tools and APIs; monitors | Progressive tool access; sandbox; kill switch | Ran at full scale — work of 853+ agents |
| Reflect | Memory (Ch. 7) | Learns and adapts from outcomes | A feedback loop that detects quality erosion | The weak link — drift not caught in time |

Read down the governance column and the theme is plain, and it answers one of this week's objectives directly: governance is not a layer you bolt onto a finished agent, it sits at every move of the loop. At Sense you govern what the agent may see and how clean it is. At Plan you govern whether the reasoning is visible and whether uncertainty is surfaced. At Act you govern which tools are live, in what order they were earned, and where the agent must stop. At Reflect you govern whether a real feedback signal exists and who reads it. An agent whose governance is concentrated at a single point — a human approving the final action, say — is governed far less than it looks, because three of the four moves have already happened unobserved by the time that approval is asked for.

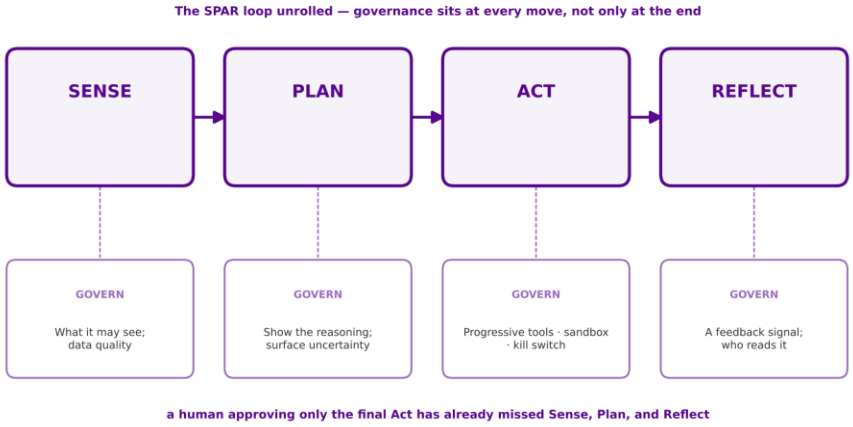



*Figure 3. The SPAR loop unrolled, with the governance question that belongs at each move. Oversight concentrated only at the final action has already missed what happened at Sense, Plan, and Reflect. SPAR loop adapted from Bornet et al. (2025), Ch. 2; the governance mapping at each stage is developed for this course.*

**Three keystones carry the four moves.** Reasoning carries Plan, Action carries Act, Memory carries Reflect; Sense is perception and leans on the same memory. Each keystone brings its own governance question — can we see the reasoning, can we contain the action, can the agent actually learn — and the thinnest keystone in a deployment is where it will fail.

## Match the Autonomy to the Risk: The Ladder, the Golden Rule, and the Kill Switch

Knowing how an agent is built does not tell you how much to let it do. That is a separate decision, and it is a governance decision, not a technical one. Return to the ladder. Level 3 is a real rung, not a ceiling — an agent that reasons, plans, and acts within a defined domain, with a human still expected to step in on the hard cases. You do not have to climb higher just because you can, and Bornet is blunt that you usually should not. His rule, in his own words, is the Golden Rule of AI agents: *the simpler, the better* (Ch. 2). Match the autonomy of the agent to the risk of the step it takes, not to the sophistication you are capable of buying.

The image to keep is Bornet's: treat the agent as a capable new hire. You would not hand a new employee every system password and let them run unsupervised on day one, however able they are; you would grant access as they earn trust, and you would keep the consequential calls under review. The same holds for an agent. This is the human-in-the-loop principle you first met in Week 5, now sharpened: the more the machine can decide, the more deliberately you must design the human's place in the decision. Human-in-the-loop is not a fallback for when the agent is weak; it is a structural choice about where judgment stays, strongest exactly where the stakes are highest — finance, healthcare, anything irreversible.

The ladder gives you the vocabulary to make that choice deliberately. Level 3, where this week lands, is an agent that reasons, plans and acts within a defined domain while a human stays available for the hard cases; Level 4 would adapt its own strategies across sessions with little oversight; Level 5, still theoretical, would set its own goals. You are not obliged to climb, and Bornet's own example is the one to copy: a financial-services firm might run Level 1 or 2 agents for transaction processing, where predictability and a clean audit trail matter most, while reserving a Level 3 agent for customer service, where adaptability earns its keep and the cost of an error is lower and more visible. Notice that this is exactly the surface Klarna chose. The choice of rung, though, is only half the decision; the other half is what you build around the rung to keep it in bounds.

Which brings the module's sharpest governance question into view, and it lives at the Act keystone, because Act is where the agent touches real systems. If an agent can execute on its own, you need a way to stop it on your own. That is the kill switch, and the question is not whether to have one but where its threshold sits and who holds the authority to pull it. At what level of autonomy should a kill switch be mandatory? Who is allowed to activate it — the operator, the manager, a compliance officer? And does your answer change with the domain: a stalled marketing agent is an inconvenience, a stalled trading agent or a mis-triaging clinical agent is something else entirely? You are not asked to settle this today. You are asked to carry it into your design, because a board will ask it, and "we'll add controls later" is the answer that loses the room.

**Autonomy is a dial, not a destination.** The Golden Rule, the simpler the better, says match an agent's autonomy to the risk of its step, and start below your ambition. Treat the agent as a capable new hire: grant tools as it earns trust, keep a human on the consequential calls, and put a kill switch where the agent acts. Deciding the kill-switch threshold and who holds it is design work, not an afterthought.

## Reflect Is the Keystone Enterprises Skip

Of the four moves, one fails first in the enterprise, and it is reliably the same one: Reflect. The reason is partly architectural and partly human. Architecturally, Reflect depends on the keystone that is hardest to build and easiest to defer — Memory, and specifically the feedback loop. Humanly, Sense, Plan and Act are visible and fundable, since you can demonstrate an agent reading, deciding, and doing, while Reflect is invisible until the day it is missing, and its payoff is

a problem that never happened, which no one celebrates. So it gets cut, and the agent ships with three strong moves and a thin fourth.

Bornet's account of an agent's inherent limitations (Ch. 3) explains why that thin fourth move is so dangerous. An agent does not understand the world; it predicts from patterns — a skilled actor delivering lines it does not feel. It has a common-sense gap: it will schedule a critical meeting at 3 a.m. or recommend an outdoor event during a storm because nothing in the rule said not to. It hallucinates: it will state a confident falsehood, cite a case that does not exist, recommend a drug that was never made. And left to itself it cannot tell that any of this has happened, because noticing requires comparing outcome to intent over time, which is exactly what Reflect, powered by a real feedback loop, is for. The first three limitations make reflection necessary; the fourth, no learning without a memory structure, is what makes it hard. An agent that cannot reflect does not get safer with scale. It gets wronger, faster, and more sure of itself.

There is a structural way to see why this is the dangerous omission. Sense, Plan and Act form an open arc — input to decision to action — and an open arc runs one way, however fast. Reflect is the move that bends the arc into a circle, feeding the result of acting back into the next round of sensing. Remove it and the agent does not merely stop improving; it loses the only mechanism by which it could notice that it has begun to fail. That is why an unreflective agent gets worse at scale rather than better: every pass carries the same confidence and none of the correction, so volume multiplies the error instead of exposing it. So the design rule is to build Reflect to the same standard as Act. If you can measure how much the agent is doing, you must be able to measure how well, and feed that measure back before the next action, or you have built a machine that is strongest exactly where it is blindest.

The cost of the missing keystone is rarely a dramatic failure; it is a slow one. An unreflective agent does not crash. It drifts, handling the easy majority well while quietly getting worse on the hard minority, and because it reports the same confidence throughout, nothing in the system raises a hand. By the time the cost shows up in a customer metric or a complaint trend, the agent has been wrong at volume for weeks. That delay is where the damage accumulates: the move that would have caught it early is the one that was cut for being hard to justify early. Reflect is insurance against a loss you cannot see accruing — exactly the kind of spending that is hardest to defend in a budget meeting and most expensive to have skipped.

**Reflect fails first.** Sense, Plan and Act are visible and fundable; Reflect is invisible until it is missing. Because an agent predicts rather than understands, hallucinates with confidence, and cannot notice its own drift without a feedback loop, the part most deployments skip is the part that keeps the agent honest at scale. Engineer Reflect in before you let the agent act alone.

## Two Customer-Service Agents: One Remembered, One Reversed

The clearest way to feel what Reflect is worth is to put two customer-service agents side by side. They sit on the same thread, Customer, in the same kind of work, and they came out opposite ways. The difference was not how well they could act. It was how well they could learn.

### The one that remembered

Bornet records a major financial-services provider that built an agent to handle investment-product inquiries, exactly the kind of work where a customer references several accounts, asks a multi-step question, and expects the thread to hold (Ch. 7). The first version had only basic short-term memory. It answered single questions well and fell apart on anything multi-step: asked to compare options, update a preference, and project a return in one conversation, it lost the earlier parts and produced fragmented, repetitive, sometimes wrong answers. Rather than bolt on more tools, the team rebuilt the memory: they segmented long conversations so the agent could focus without losing the thread, tuned what it paid attention to so the key account details and requests stayed in view, and summarised earlier turns so essential points survived while noise was discarded. Error rates fell by about 65%, and the agent could carry a genuinely complex financial conversation end to end (Ch. 7). What changed was not Act; it was the agent's ability to hold and use what it had already seen. Memory, engineered deliberately, turned a frustrating demo into a system customers could trust.

### The one that reversed

Now Klarna, the buy-now-pay-later firm, and the case the module turns on — instructive precisely because it is not a clean failure. In 2024, as it prepared to go public, Klarna rolled out an autonomous AI agent for customer service, laid off workers, and paused hiring, betting hard on the agent to carry the service function (CX Dive, Nov 2025).

The **Act** keystone ran, and by the company's own numbers it ran impressively. By late 2025 Klarna's chief executive said the agent did the work of more than 853 full-time agents, up from about 700 at the start of the year; that it handled two-thirds of customer inquiries, improved response times by 82%, and cut repeat contacts by a quarter; and that it had saved the company on the order of $60 million (CX Dive, Nov 2025, reporting Klarna's Q3 2025 earnings call). Read only that and you have a triumph. Then read what the company did next. In May 2025 Klarna turned back toward human service (Bloomberg, May 2025): the chief executive said customers should always be able to reach a person, and the firm began rehiring a flexible, on-demand service workforce. An agent that was doing the work of hundreds was deliberately walked back toward a human-plus-agent model.

Be careful here, because the public record is more nuanced than "AI failed," and this course does not let you collapse it into one. Klarna maintained there was no drop in customer satisfaction after the rollout, and in its Q3 2025 presentation put the AI assistant's satisfaction on par with human agents, citing a company net promoter score of 73. The sharper reading came from outside: a Forrester analyst called the move an over-pivot to cost containment that discounted the longer-term cost to customer experience, and noted that in automating aggressively Klarna shed service staff who carried institutional knowledge (CX Dive, Nov 2025). Those are two accounts of the

same facts, and an honest board memo reports both rather than choosing the tidier one. What is not in dispute is the shape: the agent scaled its reach fast, and the company reversed toward hybrid service, while customer-service and operations costs still rose year over year, to about $50 million in the third quarter from roughly $42 million a year earlier, even against the reported $60 million of savings. The headline saving and the rising cost line sit in the same quarter, and that tension is the governance story.

The reading this week offers is drawn from the architecture, not a verdict Klarna accepts. Sense, Plan and Act all worked: the agent read queries, decided responses, and acted at scale. The move that was thin was **Reflect** — the loop that detects quality erosion on the nuanced cases and adapts before customers feel it. The architecture predicts this exact outcome. An agent with a strong Act keystone and a weak feedback loop will scale its reach faster than it scales its ability to notice when it is getting worse; it will handle the simple two-thirds beautifully and degrade on the hard third without raising its own alarm. Set it beside the financial-services agent and the contrast is exact: that team found the same wall, good on the simple case and lost on the complex one, and answered it by engineering the memory and feedback the hard cases needed; the reversal Klarna performed by hand in May is what it looks like when a human has to supply the reflection the system could not.

The two cases also tell you why Klarna is the right one to carry into your Board Memo. First, it is customer service — the Customer thread, the surface where agents are deployed first because the volume is high and the wins are visible, and exactly where a thin Reflect loop is felt soonest, by the people least willing to tolerate it. Second, the company was relatively open about the trade: it reported the reversal and the rising cost line alongside the savings, which is what lets you reason about the case at all. Your §5 has to do the same: state what the agent will do, what it will save, and, in the same breath, where it could degrade and what you have built to catch it. A board that hears only the savings has been sold; a board that hears the savings and the stopping rule has been informed.

**Same work, opposite outcomes: the difference was Reflect.** Two customer-service agents hit the same wall: strong on simple cases, weak on complex ones. The financial-services team engineered the memory and feedback to close the gap and cut errors by about 65%; Klarna scaled Act without a matching Reflect loop and a human had to supply the correction by reversing course. The record is nuanced, not a clean failure; the structural lesson is firm — build Reflect to the standard of Act before you let an agent act alone.

## Where This Points: The Threads, the Labs, and the Capstone

On the threads, this week is where **Customer** stops being a claim and becomes the subject: the architecture you learned is the architecture of the systems being pointed at service and fulfilment first, and the governance you will design is the governance a customer-facing agent demands. Tag your own agent's value by thread as you sketch it, but expect it to land on Customer, because that is where Module 3 is heading.

On the labs, each one builds on the last. Lab 1 mapped your process and marked both kinds of decision point. Lab 2 built the stable-rules one as a bot and broke it on the judgment one, giving you, in your own data, the experience of a rule running out. Last week named that judgment step as the thing an agent takes. This week gives you the architecture of the agent that takes it — SPAR and the three keystones. And Lab 3, in Week 9, has you build that agent on a no-code platform and then attack it with deliberately difficult inputs to find where it fails. The step you could not rule in Lab 2 is the step you will build for in Lab 3, and this week is where you specify the machine that stands in it.

On the capstone, your homework is the architecture half of **Board Memo §5, Agent Design and Governance**, the piece that fills slide S8. Write a one-paragraph SPAR sketch for your Lab 3 agent: its **goal** (the single judgment step from last week, stated as an objective the agent pursues); its **tools** (the few interfaces it needs to act, named and no more, keeping to the simpler-the-better rule); and, above all, **where Reflect must sit** — the feedback signal that will tell you the agent is drifting before your customer does, and what happens when it fires. That paragraph is the seed of §5 and the direct input to your Lab 3 build.

A word on how to write it, since the paragraph is short and easy to get wrong. Lead with the goal in one sentence, stated as last week's judgment step turned into an objective the agent pursues: not "use AI for service" but "resolve a billing dispute within policy, or escalate." Name the tools next, as a short list of specific interfaces, and justify the shortness, since each tool is a door you are opening into a live system. Then spend most of your words on Reflect — the single signal you will watch to know the agent is drifting, where that signal comes from, who sees it, and what happens when it trips. If you find yourself writing a long tool list and a one-line Reflect, you have the proportions of a failed deployment on paper; turn them around before you build.

**This week specifies the machine; Lab 3 builds it.** Customer leads the module. The lab spine holds: Lab 2 broke at the judgment step, this week gives its architecture, Lab 3 builds and breaks the agent. Your homework opens Board Memo §5 and fills slide S8 — a one-paragraph SPAR sketch naming the agent's goal, its few tools, and where Reflect must sit. Aim at one step, with a stopping rule.

## What Comes Next — and What This Note Leaves for Later

Three things are handed forward on purpose; reach for them in the weeks they belong to, not this one. Lab 3 next week builds the agent on the course's chosen no-code platform; a recommended but optional companion for that build is Bornet et al. Ch. 8, which sets out a practical framework for designing an agent — read it with this week's architecture behind you if you want the fuller treatment, though it is not required to complete the lab. Multi-agent systems, what happens when several agents coordinate, delegate, and, less happily, compound one another's errors, belong to Week 10, not this week; this note treats a single agent, and you should too. And the deepest governance of autonomous systems — kill switches in force, accountability when the human has been removed from the loop, the human-AI boundary at its hardest — is developed in Week 10

and across Module 4. This week's job is complete when you can draw the SPAR loop, name the keystone under each move, and say where Reflect must sit in the agent you are about to build.

## Study Questions

Bring reasoned answers; they are designed to be argued with the case and the architecture, not recited.

1. Define the SPAR framework (Sense, Plan, Act, Reflect) and use the Klarna case to explain what happens when the Reflect component is weak or missing from the loop.
2. What are the Three Keystones of agentic AI, and why does each one introduce a distinct governance challenge?
3. If an agent can execute integration functions autonomously via the Act keystone, what governance steps must be in place before those tools are activated?

## Running Glossary

**AI agent (agentic AI).** A system that pursues a goal by deciding for itself what steps to take, built when a language model (reasoning and planning) is joined to an action layer (tools that act in real systems). Levels 3 and above on the autonomy ladder. Defined here; previewed in Week 7 (Bornet et al., 2025).

**SPAR.** Bornet's four-move architecture of an AI agent — Sense, Plan, Act, Reflect — run as a continuous loop rather than a checklist, each move feeding the next and Reflect feeding the next Sense (Bornet et al., 2025, Ch. 2).

**Sense.** The perception move: gathering data, detecting triggers, and holding working context. Bounded by the context window; leans on the memory keystone.

**Plan.** The reasoning move: weighing options and sequencing steps toward the goal rather than jumping to the first plausible action. Carried by the Reasoning keystone.

**Act.** The execution move: reaching into real systems through tools and interfaces, and monitoring the result. Carried by the Action keystone; the move where a kill switch belongs.

**Reflect.** The learning move: evaluating outcome against intent and carrying the lesson forward. Carried by the Memory keystone and, specifically, its feedback loop; the move enterprises most often underbuild.

**The three keystones.** Bornet's load-bearing agent capabilities — Action (Ch. 5), Reasoning (Ch. 6), and Memory (Ch. 7) — the capabilities beneath, respectively, Act, Plan, and Reflect.

**Context window.** An agent's working (short-term) memory — the finite span of recent information it can hold at once. What it cannot hold, it cannot act on (Bornet et al., 2025, Ch. 7).

**Procedural / episodic / semantic memory.** The three kinds of human long-term memory Bornet maps onto agents: a learned skill (how to ride a bike), a specific past event (a first interview), and a standing fact (Paris is the capital of France). An agent needs all three: learned procedure, record of what it did, and stable domain knowledge (Bornet et al., 2025, Ch. 7).

**Feedback loop (operational memory).** The mechanism that turns the outcome of an action — a correction, a complaint, a quality signal — into an adjustment, so the agent improves over time. The core of Reflect; storage without it is not reflection (Bornet et al., 2025, Ch. 7).

**Large reasoning model (LRM).** A model built for deliberate, step-by-step reasoning rather than instant pattern-matching; slower than a plain language model but more reliable on constrained, interlocking problems (Bornet et al., 2025, Ch. 6).

**Tool (function call) / progressive tool access.** A tool is an interface — an API, a function, a system login — through which an agent acts. Progressive tool access grants tools on a need-to-use basis as the agent earns trust, the way a new hire is onboarded; more tools is not a better agent (Bornet et al., 2025, Ch. 5).

**Kill switch.** A deliberate means of stopping an autonomous agent. Belongs at the Act keystone, where the agent touches real systems; its threshold and who may pull it are governance decisions that vary by domain.

**Golden Rule of AI agents.** Bornet's design maxim — "the simpler, the better" — match an agent's autonomy to the risk of its step and start below your ambition (Bornet et al., 2025, Ch. 2).

**Agentic workflow (Level 3).** The autonomy rung where an agent reasons, plans, and acts within a defined domain, with a human expected to step in on hard cases — the step this week takes up from Level 2 (Bornet et al., 2025, Ch. 2).

**Autonomy ladder (carried).** The six-rung scale from fully manual (Level 0) to fully autonomous (Level 5) that organises Module 3; Module 2 lived at Level 1, Week 7 was the step to Level 2, this week is the step to Level 3 (Bornet et al., 2025, Ch. 2).

**Judgment decision point (carried, Week 7).** A step where the rule runs out and a person — or, now, an agent — must weigh the case. The step this week's agent is built to take and the target of the Lab 3 build.

**Human-in-the-loop (carried, Week 5).** A deliberate design choice to route certain cases to a person. With an autonomous agent, the more the machine can decide, the more carefully the human's place must be designed — strongest where stakes are highest.